# AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

**Dept. of Computer Science**
**Faculty of Science and Technology**

## CSC2210: OBJECT ORIENTED PROGRAMMING 2

**Summer 2025-2026**

**Section: [*W*]**

**Group No:**

**Project Report On**

# Battery Swap Station Management System

**Supervised By**

**Al Jubair Hossain**

**Submitted By:**



| Name | ID |
|---|---|
| 1. MD. TOHIDUR RAHMAN | 25-60590-1 |
| 2. FARHAN HASEEN HAQUE | 25-60591-1 |
| 3. SAIMA SULTANA | 25-60587-1 |
| 4. A.K.M SHARIF-UL AZAD TORUN | 25-61295-1 |



| Obtained Marks for CO2 and CO4 (Description given in the following page) | | | | | |
|---|---|---|---|---|---|
| **Assessment Criteria** | **Not Attended/ Incorrect (0)** | **Inadequate (1-2)** | **Average (3)** | **Good (4)** | **Excellent (5)** |

| **Evaluation Criteria (CO2)** | **Total =** | **Evaluation Criteria (CO4)** | **Total =** |
|---|---|---|---|
| Requirement fulfillment | | Design & Innovation | |
| Validation | | Teamwork & Ethics | |
| Verification | | Communication | |

| **CO2:** Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application. | | | | | |
|---|---|---|---|---|---|
| **Assessment Criteria** | **Not Attended/ Incorrect (0)** | **Inadequate (1-2)** | **Average (3)** | **Good (4)** | **Excellent (5)** |
| **Evaluation Criteria** | **Evaluation Definition** | | | | |
| Requirement fulfillment | Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately. | Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described. | Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details. | Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details. | Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities. |
| Validation | Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied. | Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to | Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or | Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system. | Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data |

| | | | | | |
|---|---|---|---|---|---|
| | | inadequate handling of data validation. | inconsistent handling of data validation. | | validation procedures and contributing to the overall reliability and integrity of the system. |
| Verification | Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of understanding or implementation of verification processes, and data flow is not considered. | Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts are incomplete or inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality. | Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may be inconsistent or lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow. | Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and thorough, with attention to ensuring data integrity and appropriate data flow within the system. | Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system. |

**CO4:** Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

| **Assessment Criteria** | **Not Attended/ Incorrect (0)** | **Inadequate (1-2)** | **Average (3)** | **Good (4)** | **Excellent (5)** |
|---|---|---|---|---|---|
| **Evaluation Criteria** | **Evaluation Definition** | | | | |
| Requirement fulfillment | Fails to demonstrate any attempt to perform or solve system functional requirements. There is no evidence of addressing real-life scenario-based projects or identifying functional requirements for Object-Oriented | Demonstrates limited understanding of system functional requirements and real-life scenario-based projects. Attempts to perform or solve functional requirements may be incomplete or | Shows a basic understanding of system functional requirements and attempts to perform or solve them. However, the execution may lack depth or thoroughness, and there may be gaps in addressing all aspects of real- | Effectively demonstrates the performance and solution of system functional requirements for real-life scenario-based projects. Requirements are identified with clarity and relevance, and the execution is mostly accurate | Exhibits an exceptional understanding of system functional requirements and adeptly performs and solves them for real-life scenario-based projects. Requirements are meticulously identified and |

| | | | | | |
|---|---|---|---|---|---|
| | Programming project development activities. | inaccurately applied, and there is insufficient consideration given to proper identification of requirements. | life scenario-based projects or properly identifying functional requirements. | and comprehensive, reflecting a solid understanding of Object-Oriented Programming project development activities. | executed with thorough attention to detail, showcasing a comprehensive mastery of Object-Oriented Programming project development activities. |
| Validation | Fails to demonstrate any understanding or implementation of validation forms or data validation techniques. There is no evidence of addressing validation requirements or dealing with data validation in their system. | Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation. | Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation. | Demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system, although there may be minor issues. | Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system. |
| Verification | Fails to demonstrate any understanding or attempt to verify system data or functional requirements. There is no evidence of addressing verification requirements or considering data flow in their system solution. | Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts may be incomplete or inaccurately applied, and there is insufficient consideration given to ensuring proper functional requirements and data flow. | Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may lack thoroughness or precision, and there may be gaps in ensuring proper functional requirements and data flow within the system. | Effectively identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and comprehensive, with attention to ensuring data integrity and appropriate data flow within the system. | Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | throughout the system. |

## Table of Contents

# 1. INTRODUCTION

The Battery Swap Station Management System is a .NET Framework (4.7.2) Windows Forms application, built in C# with an SQL Server (LocalDB / SQL Express) backend, developed to manage the day-to-day operations of an electric-vehicle battery swapping station. Recognizing that a swap station needs to track battery stock, register electric-vehicle drivers, record billing transactions, and monitor daily performance, this project delivers a single desktop application that brings these responsibilities together under one role-based system.

The system distinguishes between two types of users: an Admin, who has full access to every module, and an Employee, whose access is restricted at the dashboard level so that battery-inventory management and reporting remain in Admin-only responsibilities. Authentication is handled through a dedicated login form that verifies credentials against the Users table and forwards the assigned role to the dashboard, which then enables or disables functionality accordingly.

As part of the system's design, an Entity-Relationship (ER) diagram was created to model the underlying data structure. This diagram includes the core entities: Users, Batteries, Transactions, and Drivers, along with the relationships that connect them through the application's data access layer. The normalization of these entities is discussed up to the Third Normal Form (3NF) in Section 3.

The following sections document the complete project scope, the feature set implemented in each module, the ER diagram with its normalization steps, the UML Use Case and Activity diagrams describing system behavior, and a summary conclusion.

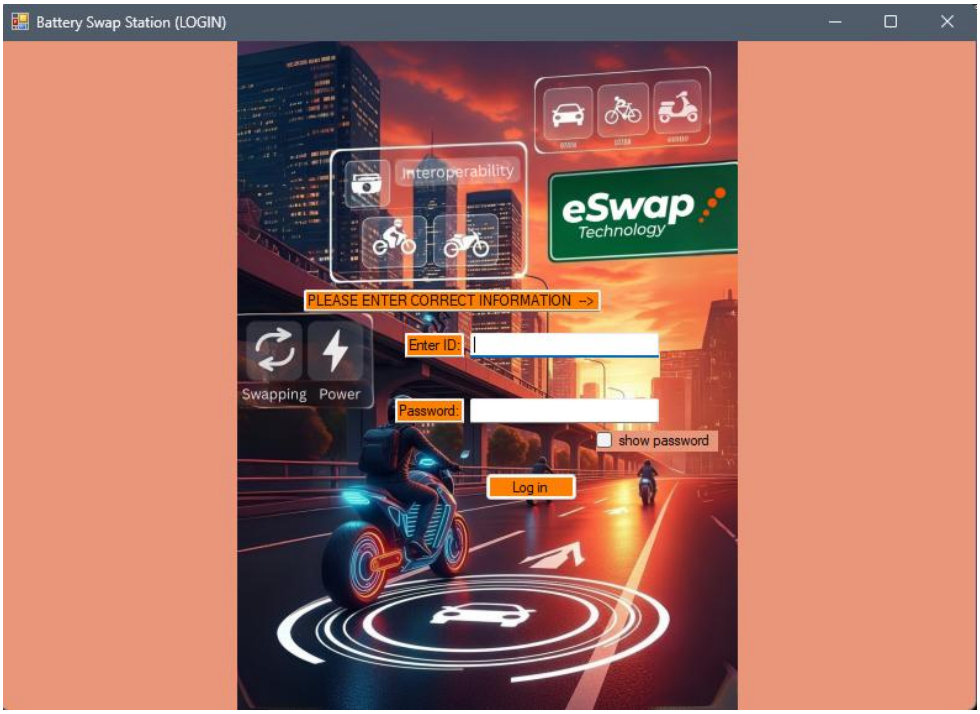


*Figure 1: Login form*

## 2. FEATURE LIST

The Battery Swap Station Management System includes the following major features, organized by module:

**1. User Authentication & Role Management**

- Secure login form validating Username and Password against the Users table
- Role-based access control distinguishing Admin and Employee accounts
- "Show Password" toggle on the login screen for user convenience
- Role-aware Dashboard that enables/disables modules depending on the logged-in user's role

**2. Battery Inventory Management**

- View all batteries in a list, showing Battery ID, Status, and Charge Level
- Add a new battery record with status (e.g., Available, In-Use, Charging) and charge level
- Update an existing battery's status/charge level by selecting it from the list
- Delete a battery record, with a confirmation prompt and protection against deleting batteries linked to existing transactions
- Input validation on charge level (must be numeric) and required fields

**3. Billing & Transaction Management**

- Generate a bill by linking a User ID and Battery ID to a new transaction with an amount and timestamp
- Referential-integrity checks that confirm the referenced User and Battery exist before a transaction is inserted
- View full transaction/billing history in a data grid, joined with Username and Battery Status for readability
- Automatic timestamping of each transaction at that moment it is generated

**4. Driver Management**

- Register drivers with Name, Phone, Card Number, Vehicle Type, and Account Balance
- Add, update, and delete driver records
- Live search/filter of drivers by name, phone, or card number as the user types
- Active/Inactive status flag per driver

**5. Reports & Analytics**

- Today's total earnings, calculated from the Transactions table
- Today's total number of battery swaps
- A chart visualising battery usage grouped by status (e.g., how many batteries are Available vs. Charging vs. In-Use)

## 2.1 Admin Role

An Admin has unrestricted access to the system. After logging in, the dashboard displays all four modules — Battery Inventory, Billing, Driver Management, and Reports — fully enabled. This role is intended for station managers who need complete oversight of stock, revenue, and driver records.


*Figure 2. Admin Role.*

## 2.2 Employee Role

An Employee has restricted access. On login, the Dashboard form checks the returned role and disables the Battery Inventory and Reports buttons, leaving Billing and Driver Management available. This reflects a front-desk staff member who processes swaps and driver registrations but does not manage stock levels or view business analytics. The "Inventory" and the "Reports" button are a bit blurred as it is un-accessible.


*Figure 2. Employee Role.*

## 3. ER DIAGRAM

This section presents the Entity-Relationship model of the system database, along with the normalization steps applied to reach the Third Normal Form (3NF), and the finalized diagram.

### 3.1 Normalization

Four core entities were identified in the data access layer: Users, Batteries, Transactions, and Drivers. Each is examined below against 1NF, 2NF, and 3NF.

- **First Normal Form (1NF)** All four tables satisfy 1NF: every column holds a single, atomic value (e.g., a battery's ChargeLevel is one integer, a driver's Phone is one value), there are no repeating groups or multi-valued columns, and every table has a defined primary key (Id for Users/Batteries/Transactions, DriverId for Drivers) that uniquely identifies each row.

- **Second Normal Form (2NF)** 2NF requires 1NF plus the elimination of partial dependencies — i.e., every non-key attribute must depend on the whole primary key. Since all four tables use a single-column primary key (no composite keys), partial dependency cannot occur, so all tables are automatically in 2NF. For example, in

Transactions, Amount and Date both depend fully on the single-column key Id, not on part of a composite key.

- **Third Normal Form (3NF)** 3NF requires 2NF plus the elimination of transitive dependencies — non-key attributes must depend only on the primary key, not on other non-key attributes. Reviewing each table:
    - **Users (Id, Username, Password, Role, Email):** Username, Password, Role, and Email each describe the user directly; none of them determines another non-key attribute, so there is no transitive dependency.
    - **Batteries (Id, Status, ChargeLevel):** Status and ChargeLevel are independent attributes of the battery; neither is derivable from the other.
    - **Drivers (DriverId, Name, Phone, CardNumber, VehicleType, Balance, IsActive):** Each attribute describes the driver directly, with no attribute depending on another non-key attribute.
    - **Transactions (Id, UserId, BatteryId, Amount, Date):** UserId and BatteryId are foreign keys referencing Users and Batteries respectively (not transitive dependencies but relationship links); Amount and Date depend only on the transaction itself.

Since no non-key attribute in any table depends on another non-key attribute, all four tables satisfy 3NF. The schema is therefore fully normalized up to 3NF, which minimizes data redundancy and update anomalies while keeping the query logic in the application straightforward.

## 3.2 Finalization

The finalized ER diagram below shows the four entities and their relationships as implemented in the application's data access layer. A User (Admin/Employee) processes many Transactions, and each Transaction is associated with exactly one Battery being swapped. The Drivers table currently operates as a standalone entity — it stores registered EV drivers and their card/balance details, but is not yet linked via foreign key to Transactions in the current implementation; this is noted as a recommended future enhancement (see Conclusion).

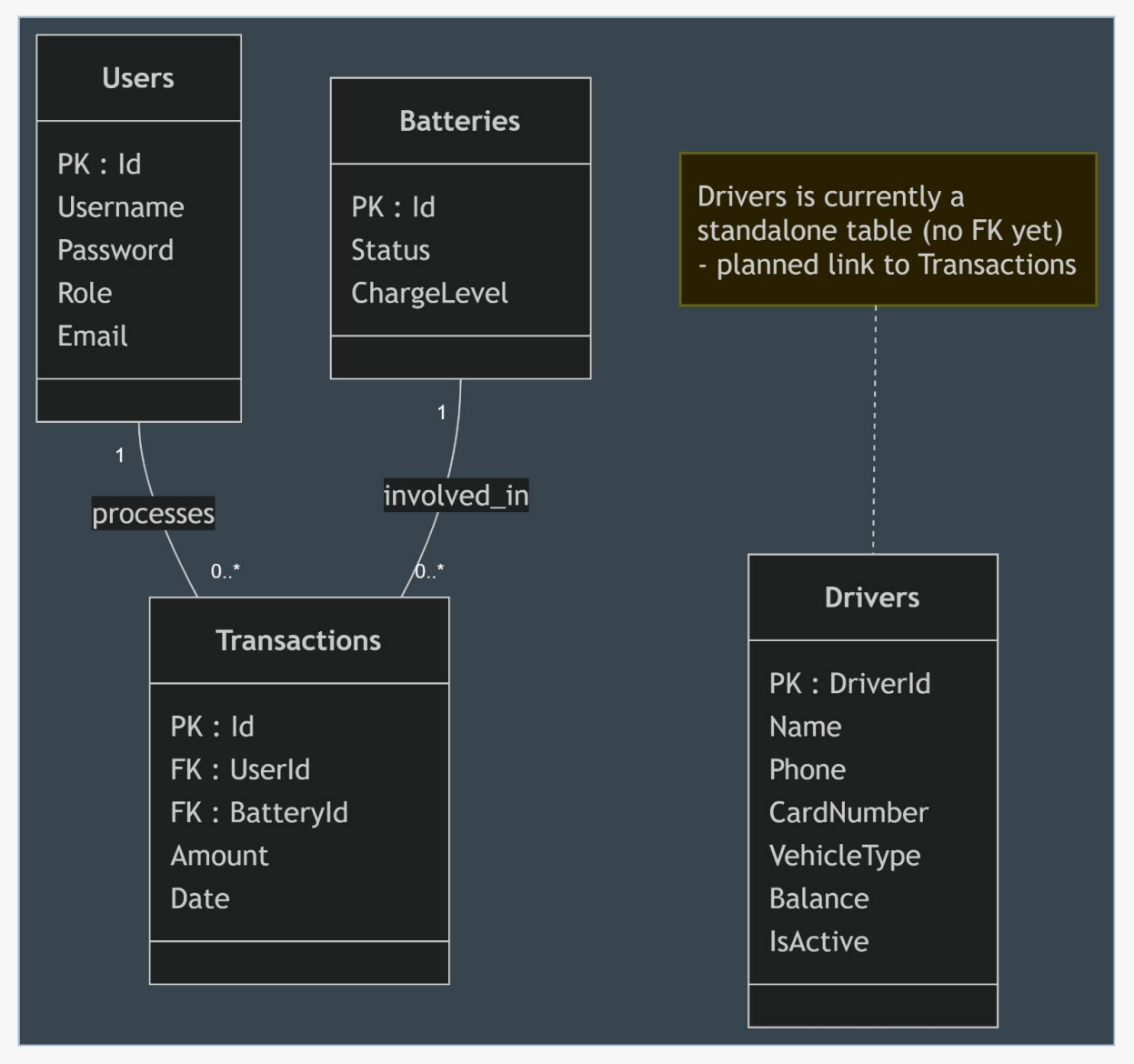


*Figure 3: Entity-Relationship Diagram (finalized, 3NF)*

# 4. UML DIAGRAM

This section presents the Use Case diagram and the Activity diagram describing the system's functional behaviour and workflow.

## 4.1 Use Case Diagram

The Use Case diagram below identifies two actors — Admin and Employee — and the system functions they can perform. The Admin has full access to all system functionalities, including Manage Battery Inventory and View Reports & Analytics, which are restricted/disabled for the Employee role at the Dashboard level. Both roles can perform core operational tasks, including Login, Logout, Generate Bill, View Billing History, and Manage Drivers. The diagram highlights

that managing battery inventory encompasses add, update, and delete operations, while managing drivers incorporates search and filtering functionality to streamline daily operations.

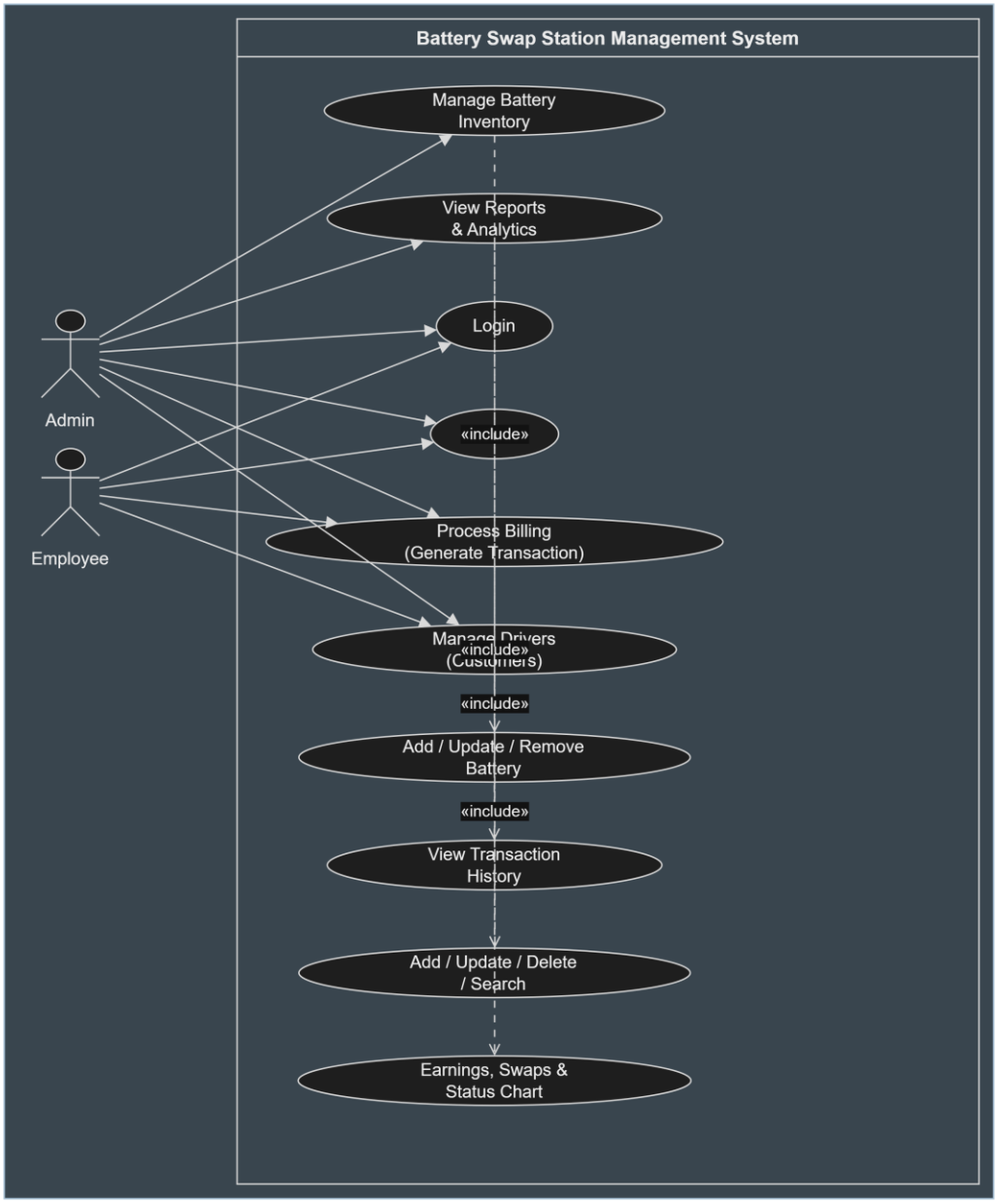


*Figure 4: Use Case Diagram*

## 4.2 Activity Diagram

The Activity diagram below traces the primary workflow of the application: a user enters their credentials, which are validated against the Users table. On failure, the login form redisplays with

an error message. On success, the Dashboard loads and immediately checks the returned role — if the role is Employee, the Inventory and Reports buttons are disabled before the user is allowed to choose a module. The user may then repeatedly select any enabled module (Inventory, Billing, Drivers, or Reports) before finally logging out, which ends the session.

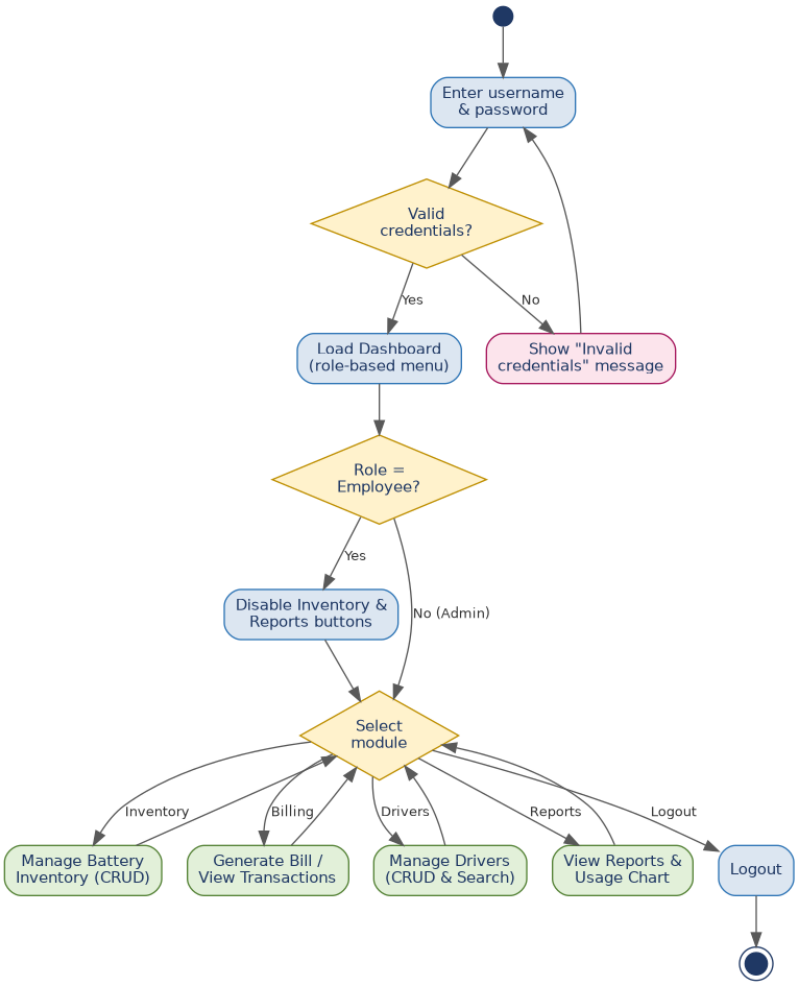


*Figure 5: Activity Diagram*

## 4.3 Class Diagram

The Class Diagram represents the structural blueprint and object-oriented design of the system, illustrating the attributes, operations (methods), and relationships among the core entities in the application layer.

- **User Class:** Manages system users (Admins and Employees) with attributes including `Id`, `Username`, `Password`, `Role`, and `Email`. It defines authentication mechanisms via

`Login()` and `Register()` methods. A single User processes multiple Transactions (1 : 1..*).

- **Battery Class:** Maintains operational attributes of swap units, such as `Id`, `Status`, and `ChargeLevel`. Each Battery is linked to transaction records over its lifecycle (1 : 1..*).
- **Transaction Class:** Serves as the central operational log, encapsulating `Id`, foreign keys `UserId` and `BatteryId`, `Amount`, and `Date`. It provides business logic methods such as `GenerateBill()` and `ViewTransactions()`.

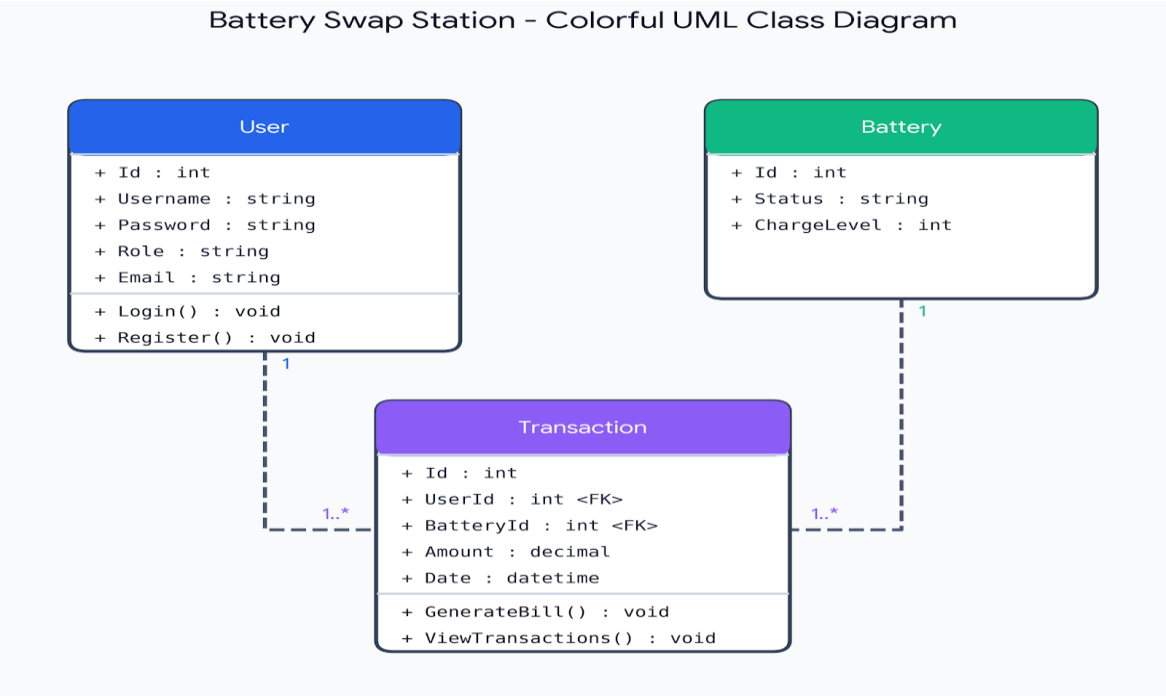


Figure 6 : UML Class Diagram

# 5. CONCLUSION

The Battery Swap Station Management System successfully delivers a functional desktop application for managing an EV battery-swap station's core operations. The project achieved its primary goals: a working role-based login and dashboard, full CRUD battery inventory management, a billing/transaction module with referential-integrity checks, driver registration with live search, and a basic reporting dashboard with a usage chart. The ER diagram, normalized up to 3NF, provides a clear view of the four core entities — Users, Batteries, Transactions, and Drivers — and how they relate through the application's data access layer, while the Use Case and Activity diagrams document the system's functional scope and primary workflow.

While the current version is fully functional for single-station use, several enhancements would make the system more robust and enterprise-ready in the future: linking the Drivers table to

Transactions via a foreign key so that swaps can be traced to a specific driver, adding a registration/sign-up form instead of relying on pre-seeded Users records, hashing stored passwords instead of storing them in plain text, adding PDF export for reports and bills, and introducing scheduled/automated reporting. All modules have been tested on Windows with the .NET Framework 4.7.2, and the codebase follows standard C# conventions with parameterised SQL queries to prevent SQL injection.